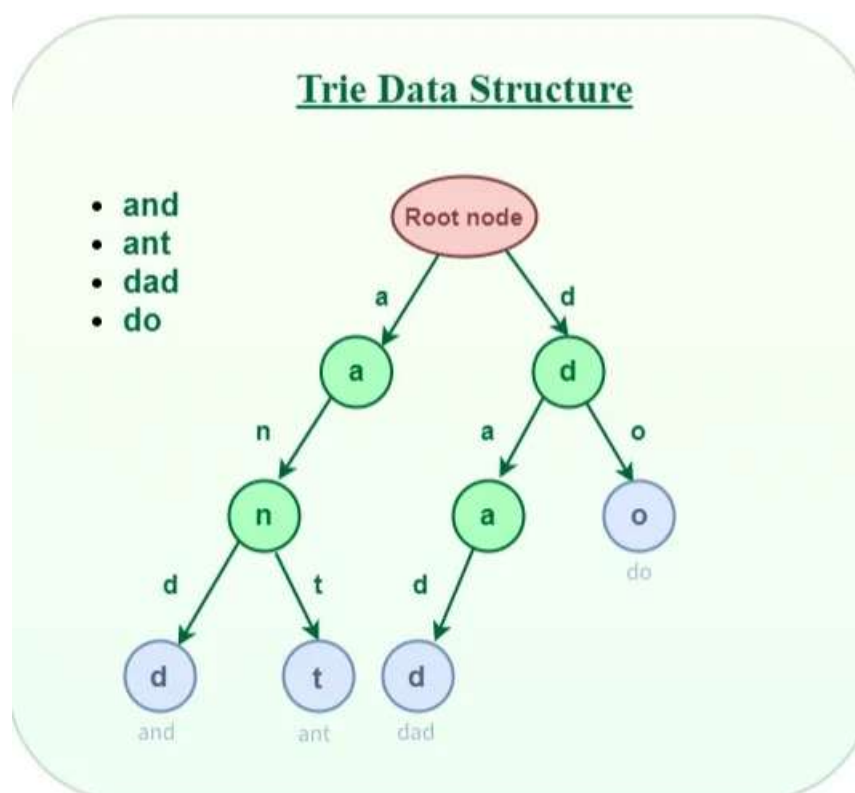


Tries

Tuesday, May 26, 2026 9:34 AM

The Trie data structure is used to store a set of keys represented as strings. It allows for efficient retrieval and storage of keys, making it highly effective in handling large datasets.

- We can insert and search strings (in a dictionary) in **$O(n)$** time where n is length of the string. This is obviously faster than BST. This is also faster than Hashing because of the ways it is implemented. We do not need to compute any hash function and no collision handling is required.
- Another advantage of Trie is, we can [easily print all words in alphabetical order](#) which is not easily possible with hashing.
- We can efficiently do [prefix search \(or auto-complete\) with Trie](#).
- The main disadvantage of tries is that they need a lot of memory for storing the strings. For each node we have too many node pointers(equal to number of characters of the alphabet)



```
class TrieNode:
    def __init__(self):
        self.children = [None] * 26
        self.isEndOfWord = False
```

Insertion in Trie Data Structure - $O(n)$ Time and $O(n)$ Space

Insert Operation in Trie Data Structure

Inserting "and" in Trie data structure:

- **Start at the root node:** The root node has no character associated with it and its **wordEnd** value is **0**, indicating no complete word ends at this point.
- **First character "a":** Calculate the index using **'a' - 'a' = 0**. Check if the **child[0]** is **null**. Since it is, create a new **TrieNode** with the character **"a"**, **wordEnd** set to **0**, and an empty array of pointers. Move to this new node.

- **Second character "n":** Calculate the index using $'n' - 'a' = 13$. Check if **child[13]** is **null**. It is, so create a new TrieNode with the character "n", **wordEnd** set to **0**, and an empty array of pointers. Move to this new node.
- **Third character "d":** Calculate the index using $'d' - 'a' = 3$. Check if **child[3]** is **null**. It is, so create a new TrieNode with the character "d", **wordEnd** set to **1** (indicating the word "and" ends here).

Method to insert a key into the Trie

def insert(root, key):

```
# Initialize the curr pointer with the root node
curr = root
```

```
# Iterate across the length of the string
for c in key:
```

```
# Check if the node exists for the
# current character in the Trie
index = ord(c) - ord('a')
if curr.children[index] is None:
```

```
# If node for current character does
# not exist then make a new node
new_node = TrieNode()
```

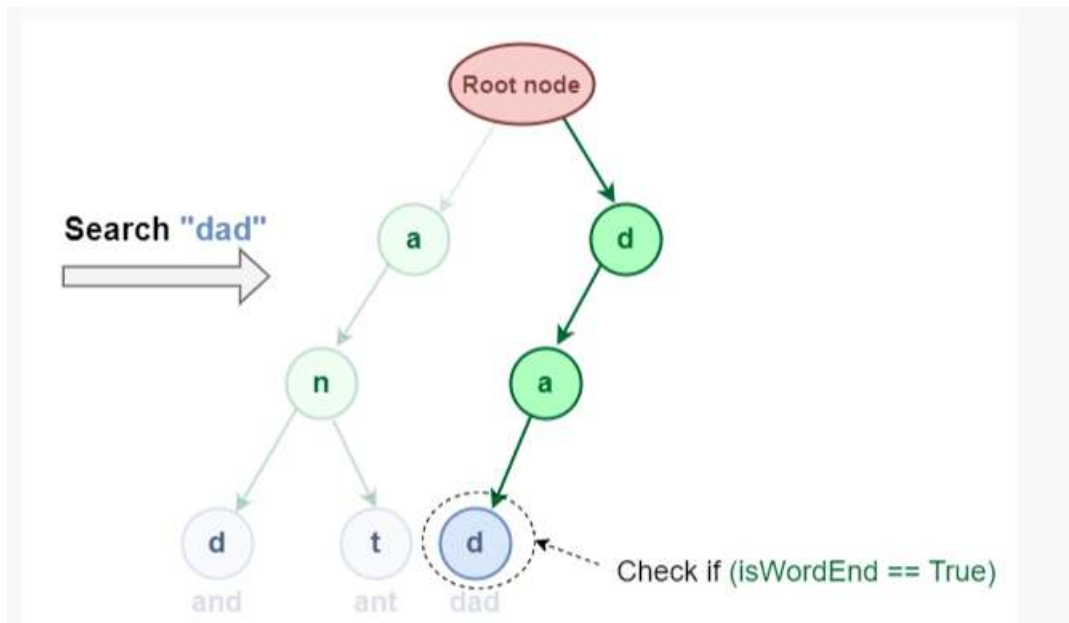
```
# Keep the reference for the newly
# created node
curr.children[index] = new_node
```

```
# Move the curr pointer to the
# newly created node
curr = curr.children[index]
```

```
# Mark the end of the word
curr.isEndOfWord = True
```

Searching in Trie Data Structure - $O(n)$ Time and $O(1)$ Space

- We start at the root node.
- We follow the branch corresponding to the character 'd'.
- We follow the branch corresponding to the character 'a'.
- We follow the branch corresponding to the character 'd'.
- We reach the end of the word and **wordEnd** flag is **1**.
- This means that **"dad"** is present in the Trie.



```
# Method to search a key in the Trie
def search(root, key):
```

```
    # Initialize the curr pointer with the root node
    curr = root
```

```
    # Iterate across the length of the string
    for c in key:
```

```
        # Check if the node exists for the
        # current character in the Trie
        index = ord(c) - ord('a')
        if curr.children[index] is None:
            return False
```

```
        # Move the curr pointer to the
        # already existing node for the
        # current character
        curr = curr.children[index]
```

```
    # Return true if the word exists
    # and is marked as ending
    return curr.isEndOfWord
```

Prefix Searching in Trie Data Structure - $O(n)$ Time and $O(1)$ Space

Searching for a prefix in a Trie data structure is similar to searching for a key, but the search does not need to reach the end of the word. Instead, we stop as soon as we reach the end of the prefix or if any character in the prefix doesn't exist in the Trie.

- We start at the root node.
- We follow the branch corresponding to the character 'd'.
- We move to the node corresponding to the character 'a'.
- We reach the end of the prefix "da". Since we haven't encountered any missing characters along the way, we return true.

```
def is_prefix(root, key):
    current = root
```

```
for c in key:
    index = ord(c) - ord('a')
```

```
# If character doesn't exist, return false
if current.children[index] is None:
    return False
current = current.children[index]
```

```
return True
```

```
class TrieNode:
    def __init__(self):
        self.children = [None] * 26
        self.isLeaf = False
```

```
class Trie:
    def __init__(self):
        self.root = TrieNode()
```

```
# Method to insert a key into the Trie
def insert(self, key):
    curr = self.root
    for c in key:
        index = ord(c) - ord('a')
        if curr.children[index] is None:
            curr.children[index] = TrieNode()
        curr = curr.children[index]
    curr.isLeaf = True
```

```
# Method to search a key in the Trie
def search(self, key):
    curr = self.root
    for c in key:
        index = ord(c) - ord('a')
        if curr.children[index] is None:
            return False
        curr = curr.children[index]
    return curr.isLeaf
```

```
# Method to check if a prefix exists in the Trie
def isPrefix(self, prefix):
    curr = self.root
    for c in prefix:
        index = ord(c) - ord('a')
        if curr.children[index] is None:
            return False
        curr = curr.children[index]
    return True
```

```
if __name__ == '__main__':
    trie = Trie()
    arr = ["and", "ant", "do", "dad"]
    for s in arr:
        trie.insert(s)
    searchKeys = ["do", "gee", "bat"]
```

```

for s in searchKeys:
    if trie.search(s):
        print("true", end= " ")
    else:
        print("false", end=" ")
print()
prefixKeys = ["ge", "ba", "do", "de"]
for s in prefixKeys:
    if trie.isPrefix(s):
        print("true", end = " ")
    else:
        print("false", end = " ")

```

Operation	Time Complexity
Insertion	O(n) Here n is the length of the string inserted
Searching	O(n) Here n is the length of the string searched
Prefix Searching	O(n) Here n is the length of the string searched

Trie | (Delete)

1. Key may not be there in trie. Delete operation should not modify trie.
2. Key present as unique key (no part of key contains another key (prefix), nor the key itself is prefix of another key in trie). Delete all the nodes.
3. Key is prefix key of another long key in trie. Unmark the leaf node.
4. Key present in trie, having atleast one other key as prefix key. Delete nodes from end of key until first leaf node of longest prefix key.

```

class TrieNode:
    def __init__(self):
        self.children = [None] * 26
        self.isEndOfWord = False

def getNode():
    pNode = TrieNode()
    pNode.isEndOfWord = False
    return pNode

def insert(root, key):
    pCrawl = root
    for i in range(len(key)):
        index = ord(key[i]) - ord('a')
        if not pCrawl.children[index]:
            pCrawl.children[index] = getNode()
        pCrawl = pCrawl.children[index]
    pCrawl.isEndOfWord = True

def search(root, key):
    pCrawl = root

```

```

    for i in range(len(key)):
        index = ord(key[i]) - ord('a')
        if not pCrawl.children[index]:
            return False
        pCrawl = pCrawl.children[index]
    return pCrawl and pCrawl.isEndOfWord

```

```

def isEmpty(root):
    for i in range(26):
        if root.children[i]:
            return False
    return True

```

```

def remove(root, key, depth = 0):
    if not root:
        return None

```

```

    if depth == len(key):
        if root.isEndOfWord:
            root.isEndOfWord = False
        if isEmpty(root):
            del root
            root = None
    return root

```

```

    index = ord(key[depth]) - ord('a')
    root.children[index] = remove(root.children[index], key, depth + 1)

```

```

    if isEmpty(root) and not root.isEndOfWord:
        del root
        root = None
    return root

```

```

if __name__ == '__main__':
    keys = ["the", "a", "there", "answer", "any", "by", "bye", "their", "hero", "heroplane"]
    root = getNode()
    for i in range(len(keys)):
        insert(root, keys[i])
    if search(root, "the"):
        print("Yes")
    else:
        print("No")
    if search(root, "these"):
        print("Yes")
    else:
        print("No")
    root = remove(root, "heroplane")
    if search(root, "hero"):
        print("Yes")
    else:
        print("No")

```

Time Complexity: The time complexity of the deletion operation is $O(n)$ where n is the key length.

Auxiliary Space: $O(n)$ due to recursive call stack, where n is the key length.

Types of Tries

A trie is a tree-like information retrieval data structure whose nodes store the letters of an alphabet. It is also known as a digital tree or a radix tree or prefix tree. Tries are classified into three categories:

1. [Standard Trie](#)

A Standard Trie has the below structure:

```
class Node {  
    // Array to store the nodes of a tree  
    Node[] children = new Node[26];  
  
    // To check for end of string  
    boolean isWordEnd;  
}
```

It is an [ordered tree](#) like data structure.

Each node(except the **root** node) in a standard trie is labeled with a character.

The children of a node are in alphabetical order.

Each node or branch represents a possible character of keys or words.

Each node or branch may have multiple branches.

The last node of every key or word is used to mark the end of word or node.

2. Compressed Trie

A Compressed Trie has the below structure:

```
class Node {  
  
    // Array to store the nodes of tree  
    Node[] children = new Node[26];  
  
    // To store the edgeLabel  
    StringBuilder[] edgeLabel = new StringBuilder[26];  
  
    // To check for end of string  
    boolean isEnd;  
}
```

A Compressed Trie is an advanced version of the standard trie.

Each nodes(except the **leaf** nodes) have at least 2 children.

It is used to achieve space optimization.

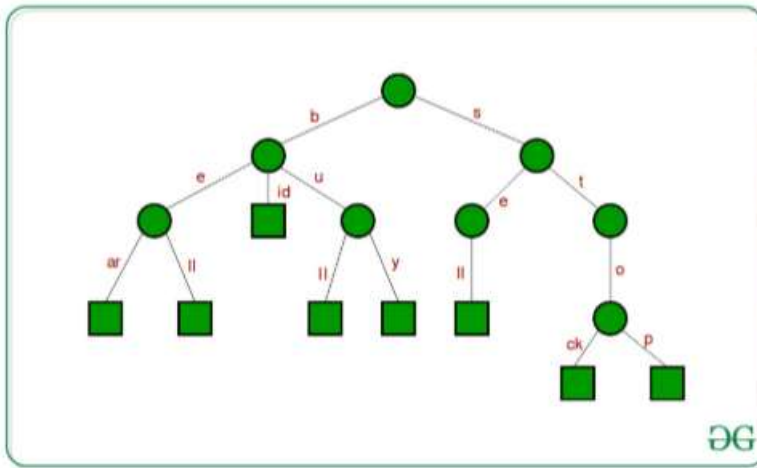
To derive a Compressed Trie from a Standard Trie, compression of chains of redundant nodes is performed.

It consists of grouping, re-grouping and un-grouping of keys of characters.

While performing the insertion operation, it may be required to un-group the already grouped characters.

While performing the deletion operation, it may be required to re-group the already grouped characters.

A compressed trie T storing s strings(**keys**) has s external nodes and O(s) total number of nodes.



3. Suffix Trie

```
struct SuffixTreeNode {
    // Array to store the nodes
    struct SuffixTreeNode *children[256];

    //pointer to other node via suffix link
    struct SuffixTreeNode *suffixLink;

    // (start, end) interval specifies the edge,
    // by which the node is connected to its
    // parent node
    int start;
    int *end;

    // For leaf nodes, it stores the index of
    // Suffix for the path from root to leaf
    int suffixIndex;
}
```

Suffix Trie is an advanced version of the compressed trie. The most common application of suffix trie is [Pattern Matching](#). While performing the insertion operation, both the word and its suffixes are stored. A suffix trie is also used in word matching and prefix matching. To generate a suffix trie, all the suffixes of given string are considered as individual words. Using the suffixes, compressed trie is built.

